



Παράλληλος και Κατανεμημένος Υπολογισμός I

Κατανεμημένη Μνήμη Μεταβίβαση Μηνυμάτων (συν. 4)

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2024



Στοιχεία υλοποίησης RPC

Στέλεχος (Stub)

- Το ενδιάμεσο λογισμικό που υλοποιεί το RPC.
- Στέλεχος πελάτη (Client stub) και στέλεχος διακομιστή (Server stub).
- Υλοποιούν τις μετατροπές των διεπαφών και τις κλήσεις, τη παράταξη / αποπαράταξη δεδομένων, την επικοινωνία με το λειτουργικό σύστημα και το επίπεδο μεταφοράς.
- Παράγονται “αυτόματα” μέσω Μεταγλωττιστών Πρωτοκόλλων (Protocol Compilers) ή βιβλιοθηκών.
- Σε πολλές υλοποιήσεις τα στελέχη παράγονται διαφανώς ή και δυναμικά κατά την εκτέλεση.

Διεπαφή (Interface)

- Διεπαφή: η υπογραφή (signature) της αποκαρυσμένης κλήσης. Πώς συντάσσεται η απομακρυσμένη κλήση; Με τι παραμέτρους και τιμές επιστροφής;
- Το συντακτικό και σημασιολογία κλήσης μπορεί να διαφέρει διαφορετικές γλώσσες. Μετατροπή μεταξύ γλωσσών.
- Γλώσσα Ορισμού Διεπαφής (Interface Definition Language, IDL): ειδική γλώσσα γενικευμένης περιγραφής διεπαφής.
- Η διεπαφή πρέπει να είναι γνωστή και στο πελάτη και στο διακομιστή.

Υλοποίηση (Implementation)

- Από τη πλευρά του διακομιστή είναι κώδικας που υλοποιεί (εκτελεί) την απομακρυσμένη κλήση.
- Στην ουσία είναι η υλοποίηση του πρωτοκόλλου εφαρμογής, το κύριο μέρος της δουλειάς μας ως προγραμματιστή της κατανεμημένης εφαρμογής.
- Από τη πλευρά του πελάτη, είναι κυρίως η επικοινωνία με το χρήστη και οι κλήσεις της απομακρυσμένης διαδικασίας.

Παράταξη (marshalling)

- Διαδικασία μετατροπής των παραμέτρων / τιμών επιστροφής της απομακρυσμένης κλήσης σε προσυμφωνημένη μορφή κατανοητή και στα δύο άκρα του RPC.
- Παράμετροι τιμής (call by value) και παράμετροι αναφοράς (call by reference) σε διαφορετικές αρχιτεκτονικές, λειτουργικά συστήματα.
- Εξωτερική Αναπαράσταση Δεδομένων (External Data Representation, XDR): τύποι και δομές δεδομένων ανεξάρτητοι από γλώσσα και αρχιτεκτονική.

Παράμετροι τιμής: παράδειγμα

3	2	1	0
0	0	0	5
7	6	5	4
L	L	I	J

(a)

0	1	2	3
5	0	0	0
4	5	6	7
J	I	L	L

(b)

0	1	2	3
0	0	0	5
4	5	6	7
L	L	I	J

(c)

(a) Αρχικό μήνυμα σε big endian.

(b) Παραλαβή μηνύματος σε little endian.

(c) Ανασύνταξη του μηνύματος ώστε να διαβαστεί σωστά.

Οι αριθμοί στα πλαίσια δηλώνουν την αρίθμηση των bytes.

Παράμετροι αναφοράς: παράδειγμα

(a) Δήλωση διαδικασίας.

Ο πίνακας `z` αντιγράφεται από το `stub`.

(b) Το τμήμα του μηνύματος με τα δεδομένα.

Ο πίνακας θα επιστραφεί ενημερωμένος και θα γραφεί πάνω στον αρχικό πίνακα `z`.

Απαιτείται συμφωνία πελάτη διακομιστή για την

αναπαράσταση σύνθετων τύπων δεδομένων.

```
foobar( char x; float y; int z[5] )  
{  
    ....  
}
```

(a)

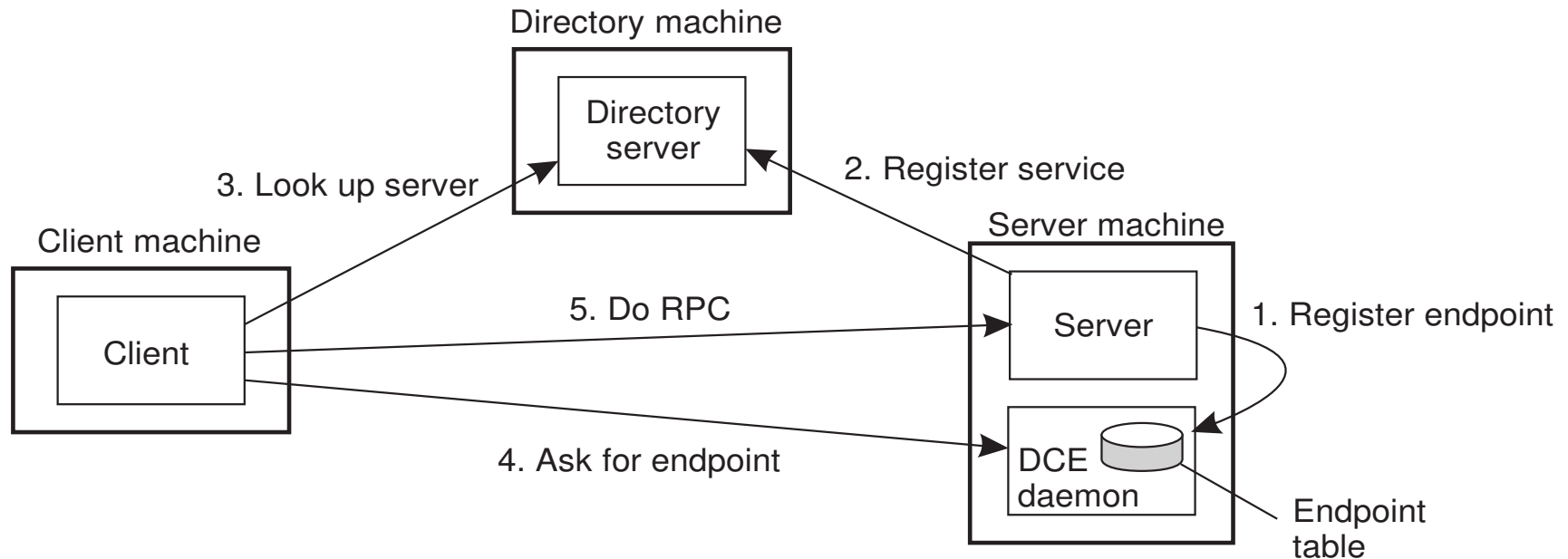
foobar's local variables	
	x
y	
5	
z[0]	
z[1]	
z[2]	
z[3]	
z[4]	

(b)

Σύνδεση πελάτη διακομιστή

- Συνήθως η επικοινωνία πελάτη διακομιστή επιτυγχάνεται μέσω υποδοχών αλλά υπάρχουν και άλλες λύσεις.
- Στατική σύνδεση: κατά τη παραγωγή των εκτελεσίμων οι ονομασίες (υποδοχές διακομιστή, ονομασίες διαδικασιών) καταγράφονται στο κώδικα.
- Δυναμική σύνδεση 1: κατά τη παραγωγή των εκτελεσίμων, καταγράφονται στο κώδικα μόνο η διεύθυνση του καταλόγου και δημόσια ονόματα διαδικασιών. Ακολουθεί φάση αναζήτησης και σύνδεσης.
- Δυναμική σύνδεση 2: το στέλεχος του πελάτη μεταφέρεται από το διακομιστή κατά την εκτέλεση ή παράγεται κατά την αρχική σύνδεση.

Τυπικό RPC: δυναμική σύνδεση



- Ο διακομιστής καταχωρεί (register, 1) την απομακρυσμένη αναφορά κάθε διαδικασίας και τη συνδέει (bind, 2) με ένα δημόσιο όνομα σε μια υπηρεσία καταλόγου RPC (directory) .
- Ο πελάτης αρχικά γνωρίζει μόνο την ονομασία του καταλόγου ή και το δημόσιο όνομα της απομακρυσμένης διαδικασίας. Μέσω του ευρετηρίου και με αναζήτηση ονόματος (lookup, 3), λαμβάνει την την απομακρυσμένη αναφορά (σημείο εισόδου / entry point, 4) της απομακρυσμένης διαδικασίας.
- Στη συνέχεια εκτελεί το RPC, 5.

Πελάτης RPC

Η εφαρμογή RPC είναι οργανωμένη σε πελάτη και διακομιστή. Συνήθως είναι δύο διαφορετικά προγράμματα.

Ο πελάτης διαθέτει:

- Τη διεπαφή (interface) που περιέχει τις υπογραφές των δημόσιων διαδικασιών.
- Το σώμα (body) που περιέχει:
 - τη σύνδεση με το διακομιστή
 - τις κλήσεις απομακρυσμένης διαδικασίας μέσω του stub.

Διακομιστής RPC

Ο διακομιστής διαθέτει:

- Τη διεπαφή (interface) που περιέχει τις υπογραφές των δημόσιων διαδικασιών.
- Την υλοποίηση (implementation) που περιέχει:
 - τοπικές (ιδιωτικές) μεταβλητές
 - την υλοποίηση των δημόσιων διαδικασιών
 - άλλες ιδιωτικές διαδικασίες με την υλοποίησή τους
- Το σώμα (body) που περιέχει:
 - τη δημοσιοποίηση διαδικασιών
 - την αναμονή και διαχείριση απομακρυσμένων κλήσεων μέσω του stub.

Ταυτοχρονισμός διακομιστή RPC

- Κάθε νέα απομακρυσμένη κλήση προκαλεί στο διακομιστή τη δημιουργία και εκτέλεση νέου νήματος / διεργασίας για την εξυπηρέτησή του.
- Σε ορισμένες υλοποιήσεις κάθε απομακρυσμένη κλήση λειτουργεί εντελώς ανεξάρτητα. Ο διακομιστής δεν έχει μοιραζόμενη κατάσταση.
- Σε άλλες υλοποιήσεις ο διακομιστής λειτουργεί ως επόπτης, δηλαδή τα νήματα εκτελούν διαδικασίες σε αμοιβαίο αποκλεισμό (αλλά απαιτείται ρητός συγχρονισμός σε συνθήκες).
- Σε άλλες υλοποιήσεις RPC απαιτείται ρητά τόσο αμοιβαίος αποκλεισμός όσο και συγχρονισμός σε συνθήκες.



Μια αφηρημένη υλοποίηση RPC

```
include RPClib;  
interface bufferInterface {  
    put(int);  
    int get();  
}
```

← Βιβλιοθήκη και Διεπαφή

RPC παραγωγός καταναλωτής Σκελετός διακομιστή 1

```
Server {
```

```
    PORT = ...
```

```
    size = ...
```

```
    buffer = new bufferImplementation(size);
```

Αναφορά στην υλοποίηση

```
    // Publish, Bind, Listen RPC enty points
```

```
    RPC.Register (buffer, PORT);
```

```
    while (true) {
```

```
        // Accept RPC calls
```

```
        RPC.Accept(buffer);
```

Δημοσιοποίηση
κλήσεων μέσω καταλόγου

```
        // Dispatch a Server Thread / Process for  
        // each RPC call executing implementation  
        // procedures put() or get()
```

Κλήση stub
διακομιστή

```
    }
```

```
}
```

← Ένα νήμα ή διεργασία ανά
εισρχόμενη κλήση

```
include RPClib;  
implementation bufferImplementation {
```

```
    // Shared State  
    int contents[size];  
    Lock l;  
    Condition bufferEmpty;  
    Condition bufferFull;  
    int size,  
        front = 0,  
        back = size - 1,  
        counter = 0;
```

RPC παραγωγός
καταναλωτής
Σκελετός διακομιστή
Υλοποίηση 1

```
procedure put(int data){  
    lock(l);  
    while (counter == size) await(l, bufferFull);  
    back = (back + 1) % size;  
    contents[back] = data;  
    counter++;  
    signal_all (bufferEmpty);  
    unlock(l);  
}
```

...

RPC παραγωγός καταναλωτής

Σκελετός διακομιστή Υλοποίηση 2

```
...  
procedure int get() {  
    int data;  
    lock(l);  
    while (counter == 0) await(l, bufferEmpty);  
    data = contents[front];  
    front = (front + 1) % size;  
    counter--;  
    signal_all (bufferFull);  
    unlock(l);  
    return data;  
}  
  
}
```

Για πλήρη κώδικα σε Java δείτε

<https://github.com/misko321/producers-consumers-rmi>

RPC παραγωγός καταναλωτής

Σκελετός πελάτη 1

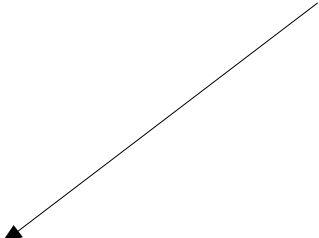
```
include RPClib;  
interface bufferInterface {  
    put(int);  
    int get();  
}
```

Βιβλιοθήκη και Διεπαφή



```
Producer {  
    HOST = ...  
    PORT = ...  
    remoteInterface = (bufferInterface)  
                        RPC.Connect(HOST, PORT);
```

Απομακρυσμένη αναφορά
Λήψη μέσω Καταλόγου



Σύνδεση με Κατάλογο
και αίτημα για λήψη
αναφοράς



```
    for (int i=0; i<noIters; i++) {  
        //... produce item  
        remoteInterface.put(item);  
    }
```

Κλήση απομακρυσμένης
διαδικασίας



```
}
```

RPC παραγωγός καταναλωτής

Σκελετός πελάτη 2

```
include RPClib;  
interface bufferInterface {  
    put(int);  
    int get();  
}  
  
Consumer {  
    HOST = ...  
    PORT = ...  
    remoteInterface = (bufferInterface)  
                        RPC.Connect (HOST, PORT);  
  
    while (true) {  
  
        item = remoteInterface.get();  
        // ... consume item  
  
    }  
}
```

Κλήση απομακρυσμένης διαδικασίας Κατάλογος (RPC Directory)

- Πρώτος ξεκινά ο διακομιστής και δημοσιοποιεί (RPC.Register) στην τοπική υπηρεσία Καταλόγου (RPC Directory) τις διεπαφές που διαθέτει.
- Κάθε διεπαφή συσχετίζεται (α) με μια υλοποίηση (bufferImplementation) και (β) με ένα σημείο εισόδου (entry point), πχ ένα αριθμό υποδοχής.
- Ο διακομιστής αναμένει αιτήματα (RPC.Accept) δηλαδή απομακρυσμένες κλήσεις στις διεπαφές μέσω των σημείων εισόδου.

Κλήση απομακρυσμένης διαδικασίας

Λειτουργία 1

Στο παράδειγμα υποθέτουμε ότι ο διακομιστής RPC:

- διατηρεί μοιραζόμενη κατάσταση: ο buffer μοιράζεται μεταξύ των νημάτων / διεργασιών. Θα μπορούσαμε να έχουμε ξεχωριστό buffer ανά κλήση.
- η λέξη κλειδί procedure δεν έχει λειτουργία επόπτη: απαιτείται ρητή χρήση κλειδώματος.
- παρόμοια, θεωρούμε ότι απαιτείται ρητή χρήση μηχανισμών ταυτοχρονισμού.

Κλήση απομακρυσμένης διαδικασίας

Λειτουργία 2

- Κάθε νέα απομακρυσμένη κλήση put ή get προκαλεί στο διακομιστή τη δημιουργία και εκτέλεση νέου νήματος / διεργασίας για την εξυπηρέτησή του, που τερματίζεται με την εξυπηρέτηση της απομακρυσμένης κλήσης.
- Κατά τη μεταγλώττιση του κώδικα πελάτη και διακομιστή παράγονται 'αυτόματα' τα stubs τα οποία είναι απαραίτητα για την εκτέλεση (RPClib).

Κλήση απομακρυσμένης διαδικασίας

Λειτουργία 3

- Ο πελάτης αρχικά απευθύνεται στο απομακρυσμένο RPC.Directory (HOST, PORT) όπου αναζητά την απομακρυσμένη διεπαφή `bufferInterface`.
- Το RPC.Directory του επιστρέφει την απομακρυσμένη αναφορά στη διεπαφή (`remoteInterface`) δηλαδή το σημείο εισόδου όπου πρέπει να αποσταλεί το αίτημα.
- Το αίτημα έχει τη μορφή κλήσης `put` ή `get`. Τα stubs του πελάτη και του διακομιστή αναλαμβάνουν την παράταξη, μεταφορά, αποπαράταξη, κλήση, και την αντίστοιχη επιστροφή.



Παραδείγματα υλοποιήσεων RPC

Υλοποιήσεις RPC 1*

- Αρχική υλοποίηση: rpcgen (Unix/Linux, C)

Σήμερα υπάρχουν δύο βασικές προσεγγίσεις:

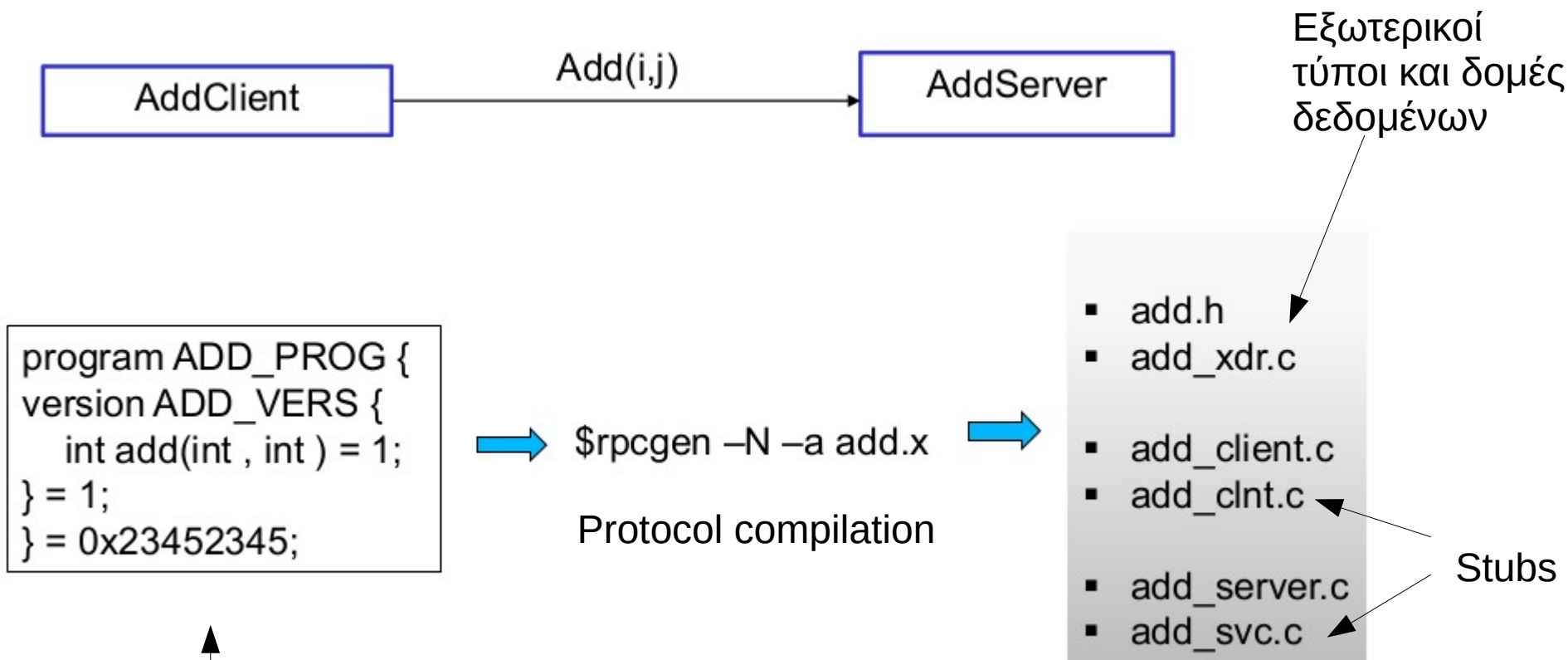
- Frameworks :
πχ Google gRPC, ApacheThrift, CORBA, MSRPC
- Βιβλιοθήκες σε γλώσσες προγραμματισμού:
πχ Java RMI, Python PyRO4, Ada, Erlang, Go..

* Περισσότερα στο μάθημα των Κατανεμημένων Συστημάτων.

Υλοποιήσεις RPC 2

- Τα frameworks είναι ιδιαίτερα ισχυρά περιβάλλοντα, υποστηρίζουν πολλές γλώσσες προγραμματισμού, αλλά δεν συνεργάζονται μεταξύ τους. Απαιτούν εγκατάσταση ενδιάμεσου λογισμικού.
- Οι βιβλιοθήκες γλωσσών είναι πολύ πιο απλές γιατί μειώνεται η ετερογένεια πελάτη και διακομιστή. Ο προγραμματισμός είναι απλούστερος αφού προσαρμόζεται στη συγκεκριμένη γλώσσα.
- Ειδικά με υλοποίηση σε εικονική μηχανή αποφεύγονται πολλά προβλήματα ετερογένειας στην αρχιτεκτονική και το λειτουργικό σύστημα.
- Δέσμευση σε μια γλώσσα προγραμματισμού.

Παράδειγμα rpcgen 1



Η διεπαφή (Interface) της απομακρυσμένης κλήσης σε IDL:
Ζητούμε από το rpcgen να παράγει κώδικα για τον πελάτη και διακομιστή RPC που να εκτελεί μια απομακρυσμένη πρόσθεση.

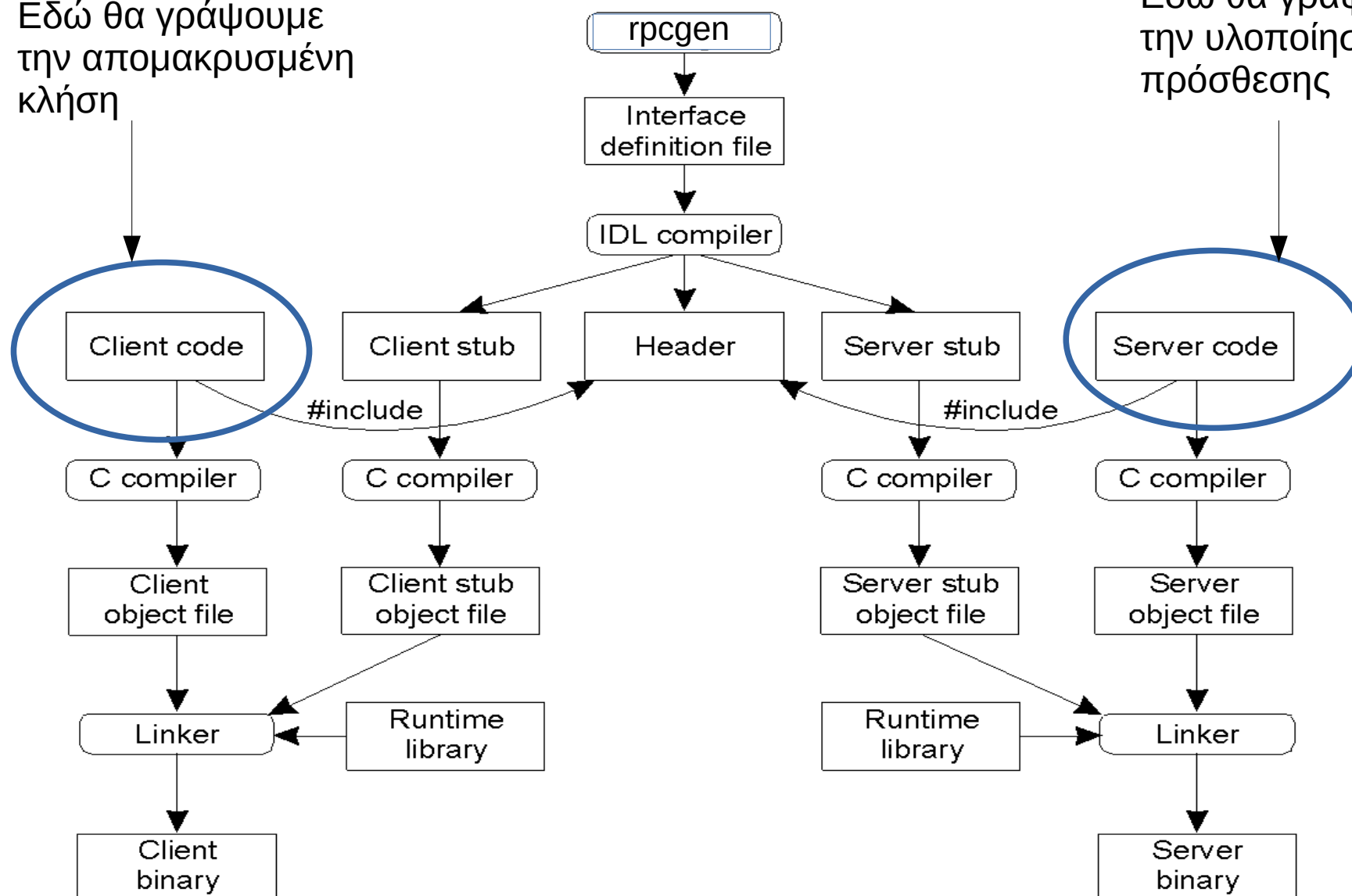
Οι κώδικες παράγονται αυτόματα.

Προσθέτουμε κώδικα στο
add_client.c
add_server.c

Παράδειγμα rpcgen 2

Εδώ θα γράψουμε
την απομακρυσμένη
κλήση

Εδώ θα γράψουμε
την υλοποίηση της
πρόσθεσης





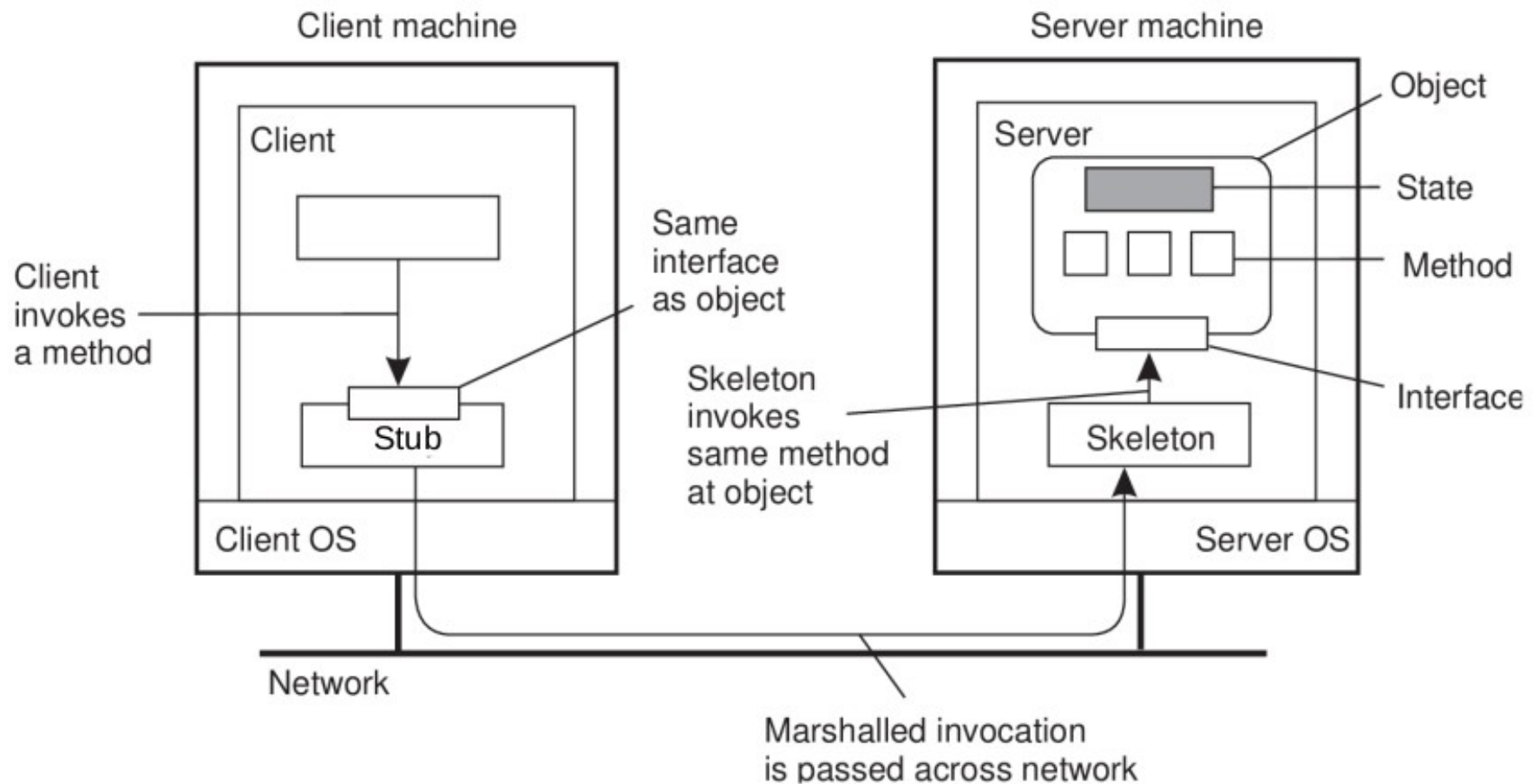
Κλήση απομακρυσμένης μεθόδου

Remote method invocation (RMI)

Java RMI

- Το πακέτο `java.rmi` αποτελεί την υλοποίηση του RPC στη Java.
- Στην Java, οι `non-static` μέθοδοι πρέπει να κληθούν επί ενός αντικειμένου.
- Έτσι η Java απαιτεί οι μέθοδοι (διαδικασίες του RPC) αλλά και η απομακρυσμένη κλήση τους να βασίζεται σε κάποιο απομακρυσμένο αντικείμενο.
- Επομένως στη Java μιλούμε για απομακρυσμένα αντικείμενα (`remote objects`), αντί για απομακρυσμένες διαδικασίες.

Κατανεμημένα αντικείμενα

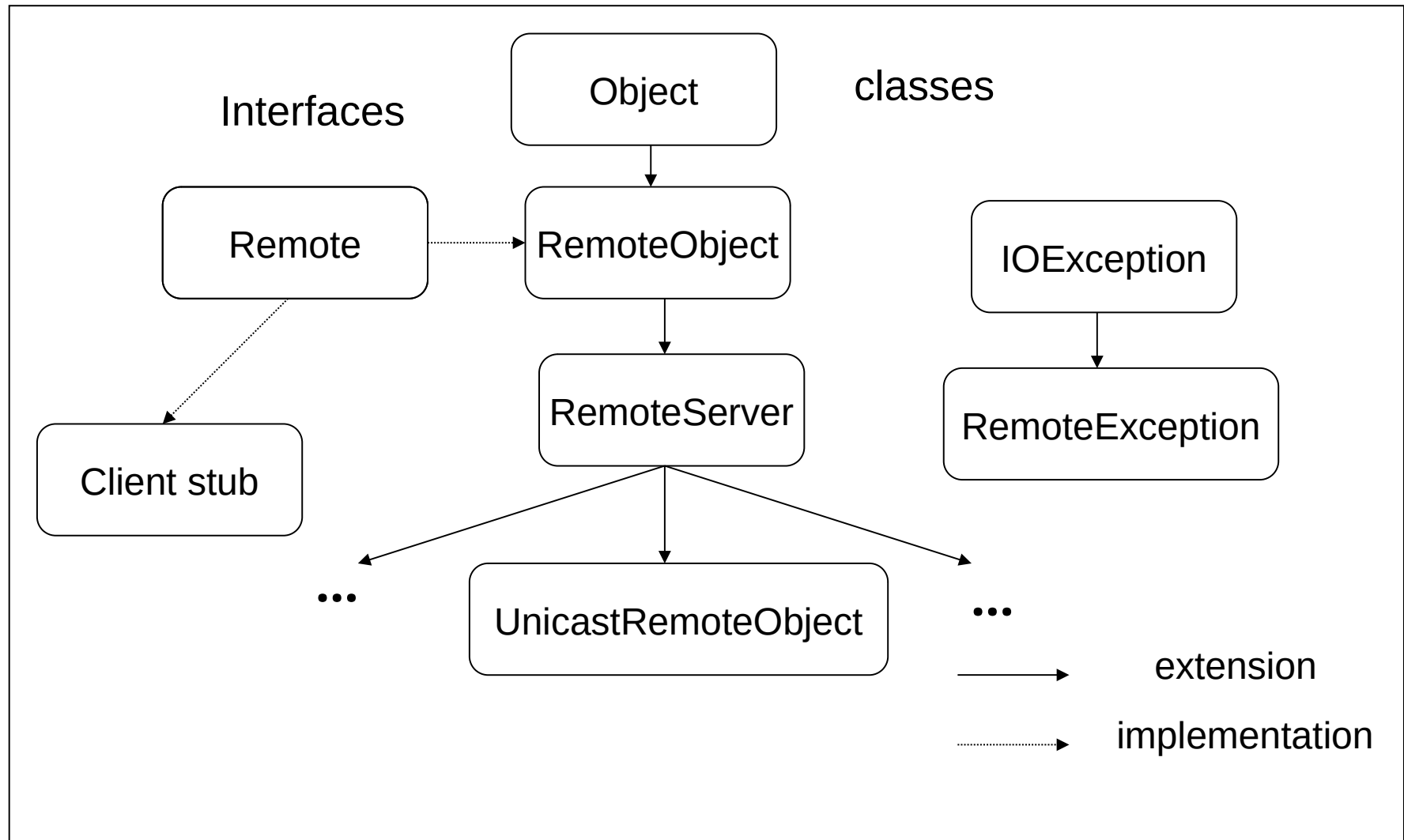


Alternative names: Stub=Proxy, Client Stub, Skeleton= Server Stub, Object adapter
Remote methods 'belong' to a remote object.

Απομακρυσμένα αντικείμενα Java 1

- Οι μέθοδοί τους μπορούν να κληθούν από διαφορετική JVM, που εκτελείται στον ίδιο ή διαφορετικό υπολογιστή.
- Περιγράφονται από μια ή περισσότερες απομακρυσμένες διεπαφές (remote interfaces) που επεκτείνουν τη διεπαφή `java.rmi.Remote`.
- Οι μέθοδοι που δηλώνονται σε μια απομακρυσμένη διεπαφή μπορεί να προκαλούν `RemoteExceptions`
- Η κλήση απομακρυσμένης μεθόδου (RMI) είναι η κλήση μιας μεθόδου η οποία είναι δηλωμένη σε μια απομακρυσμένης διεπαφή επί ενός απομακρυσμένου αντικειμένου.

Απομακρυσμένα αντικείμενα Java 2



RPC και RMI 1

- Η απομακρυσμένη διεπαφή (Interface) αντιστοιχεί στις υπογραφές των δημόσιων διαδικασιών του RPC.
- Οι κλάσεις που υλοποιούν τις απομακρυσμένες διεπαφές αντιστοιχούν στις υλοποιήσεις (Implementation) των διαδικασιών στο σώμα του RPC.
- Οι απομακρυσμένες κλήσεις από RMI Clients δημιουργούν νήματα -ανά κλήση (πολυνηματική εκτέλεση) που εκτελούνται στον RMI server.

RPC και RMI 2

- Η επικοινωνία υλοποιείται με χρήση Υποδοχών (Java Sockets).
- Η παράταξη / αποπαράταξη επιτυγχάνονται με χρήση Σειριοποίησης (Java Serialization). Δηλαδή `writeObject()` και `readObject()` όπου τα objects είναι `Serializable`.

Stub and Skeleton 1

Αρχεία εισόδου (διαθέσιμα για τη μεταγλώττιση):

- Για τον πελάτη:

Εφαρμογή RMI Client, Interface

- Για τον διακομιστή:

Εφαρμογή RMI Server, Interface, Implementation

```
$ javac rmiclient.java interface.java
```

```
$ javac rmiserver.java interface.java  
implementation.java
```

Stub and Skeleton 2

- Ο Skeleton είναι ενσωματωμένος στη κλάση του RMI server
- Ο Stub παράγεται δυναμικά από τον RMI server κατά τη λειτουργία και μεταφορτώνεται στον RMI Client

Μετά τη μεταγλώττιση είναι διαθέσιμες οι κλάσεις:

- Στο πελάτη: `interface.class`
`rmiclient.class`
- Στο διακομιστή: `interface.class`
`rmiserver.class` `implementation.class`

Μητρώο RMI (rmiregistry) 1

- Το μητρώο RMI είναι το αντίστοιχο του Ευρετηρίου (Directory) του RPC.
- Το μητρώο RMI πρέπει να βρίσκεται σε λειτουργία πριν ξεκινήσει ο διακομιστής (και ο πελάτης).
- Η διεύθυνση του μητρώου RMI είναι η μόνη αρχικά απαραίτητη για τον πελάτη RMI.
- Συνήθως το μητρώο RMI εκτελείται στο ίδιο υπολογιστή με το διακομιστή RMI.

Μητρώο RMI (rmiregistry) 2

- Η διεργασία του μητρώου RMI είναι ξεχωριστή από το διακομιστή RMI. Είναι ένας δαίμονας που 'ακούει' στη θύρα 1099. Μπορούμε αν θέλουμε να δηλώσουμε διαφορετική θύρα κατά την εκκίνηση.

Η εκκίνηση του μητρώου RMI μπορεί να γίνει:

- Ανεξάρτητα από την εκτέλεση της εφαρμογής από τη γραμμή εντολών με την εντολή
`$ rmiregistry`
- Μέσω της εφαρμογής του διακομιστή RMI, στην αρχή της εκτέλεσής της.

Μητρώο RMI (rmiregistry) 3

- Όταν ένας διακομιστής δημοσιοποιεί ένα απομακρυσμένο αντικείμενο στο μητρώο RMI, συνδέει το όνομα του αντικειμένου με ένα Stub που μεταφορτώνεται και αυτό από τον διακομιστή στο μητρώο RMI.
- Όταν ένα απομακρυσμένο αντικείμενο καταγραφεί στο μητρώο RMI τότε οι πελάτες μπορούν το αναζητήσουν στο μητρώο με βάση το όνομά του και να λάβουν ένα αντίγραφο του Stub που εξασφαλίζει την επικοινωνία με το συγκεκριμένο απομακρυσμένο αντικείμενο στο διακομιστή.
- Στη συνέχεια η επικοινωνία γίνεται απ' ευθείας μεταξύ πελάτη και διακομιστή.

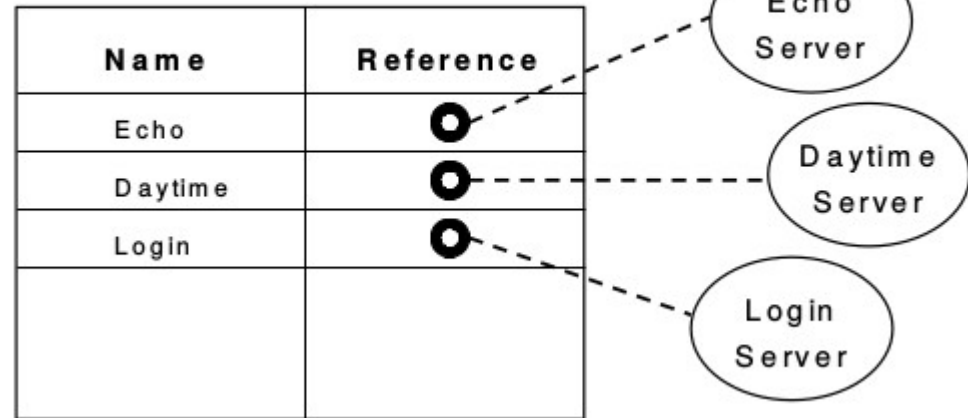
Μητρώο RMI (rmiregistry) 4

- Ονομασία απομακρυσμένης μεθόδου (αντικειμένου)

rmi://Host/Name ή

//Host:Port/Name

Μέθοδοι



- static Registry createRegistry(int port) Δημιουργία registry
- static Registry getRegistry(String host, int port) Επιστροφή αναφοράς σε registry
- void bind(String name, Remote obj) Σύνδεση απομακρυσμένης αναφοράς με όνομα στο registry.
- void rebind(String name, Remote obj) Αντικατάσταση απομακρυσμένης αναφοράς με βάση το όνομα στο registry.
- void unbind(String name) Διαγραφή γραμμής στο registry με βάση το όνομα.
- String[] list() Λίστα διαθέσιμων ονομάτων στο registry
- Remote lookup(String name) Επιστροφή απομακρυσμένης αναφοράς από registry με βάση το όνομα.

Απλό παράδειγμα RMI 1

```
// Interface
import java.rmi.*;
public interface Addition extends Remote {
    public int add (int a, int b) throws RemoteException;
}

// Implementation
import java.rmi.*;
import java.rmi.server.*;
public class AdditionImpl extends UnicastRemoteObject
    implements Addition {
    // Constructor
    public AdditionImpl() throws RemoteException {
        super();
    }

    // Code
    public int add (int a, int b) throws RemoteException {
        return (a+b);
    }
}
```

Απλό παράδειγμα RMI 2

```
import java.rmi.registry.*;
import java.rmi.server.*;

public class AdditionServer {

    private static final String HOST = "localhost";
    private static final int PORT = Registry.REGISTRY_PORT; // 1099

    public static void main(String[] args) throws Exception {

        // Define server hostname
        System.setProperty("java.rmi.server.hostname", HOST);

        // Object creation
        Addition robj = new AdditionImpl();

        // Create or locate rmiregistry, register and bind object
        // Server passes a copy of the stub to rmiregistry
        Registry registry = LocateRegistry.createRegistry(PORT);
        // Registry registry = LocateRegistry.getRegistry(HOST, PORT);
        String rmiObjectName = "MyAddition";
        registry.bind (rmiObjectName, robj);
    }
}
```

Απλό παράδειγμα RMI

3

```
// RMI Client
import java.rmi.registry.*;
import java.rmi.*;
public class AdditionClient
{

    private static final String HOST = "remotehostname";
    private static final int PORT = Registry.REGISTRY_PORT; // 1099

    public static void main(String[] args) {
        try {
            // Client looks up at rmiregistry for remote object
            // and receives the appropriate stub
            Registry registry = LocateRegistry.getRegistry(HOST, PORT);

            String rmiObjectName = "MyAddition";
            Addition ref = (Addition)registry.lookup(rmiObjectName);

            // Remote method invocation
            int result = ref.add (3, 5);
            System.out.println("The result is " + result);
        }
        ...
    }
}
```

Απλό παράδειγμα RMI 4

```
...
    catch (RemoteException re) {
        System.out.println("Remote Exception");
        re.printStackTrace();
    }
    catch (Exception e) {
        System.out.println("Other Exception");
        e.printStackTrace();
    }
}
}
```

Μεταγλώττιση σε ξεχωριστούς καταλόγους:

```
$javac Addition.java AdditionImpl.java AdditionServer.java
```

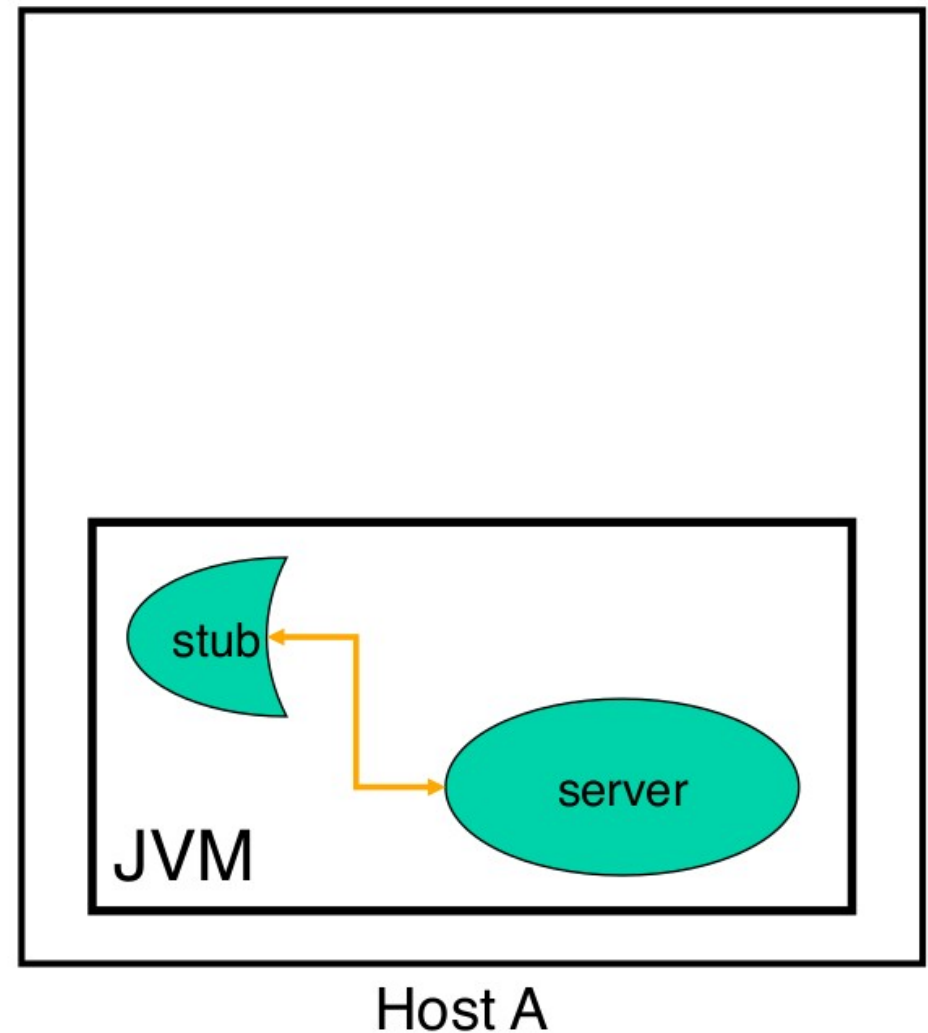
```
$javac Addition.java AdditionCilent.java
```

Code/RMIadd

Εκτέλεση παραδείγματος: διακομιστής 1

```
Addition robj = new  
AdditionImpl();
```

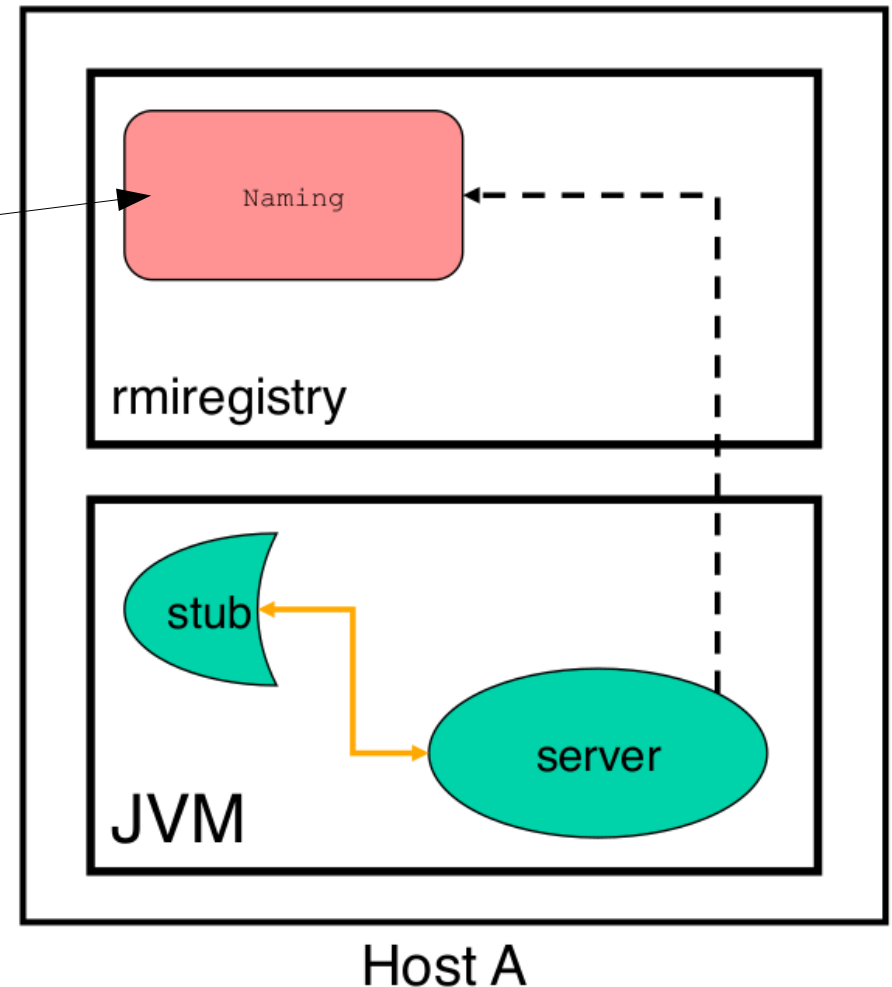
Αρχικά δημιουργείται το stub



Εκτέλεση παραδείγματος: διακομιστής 2

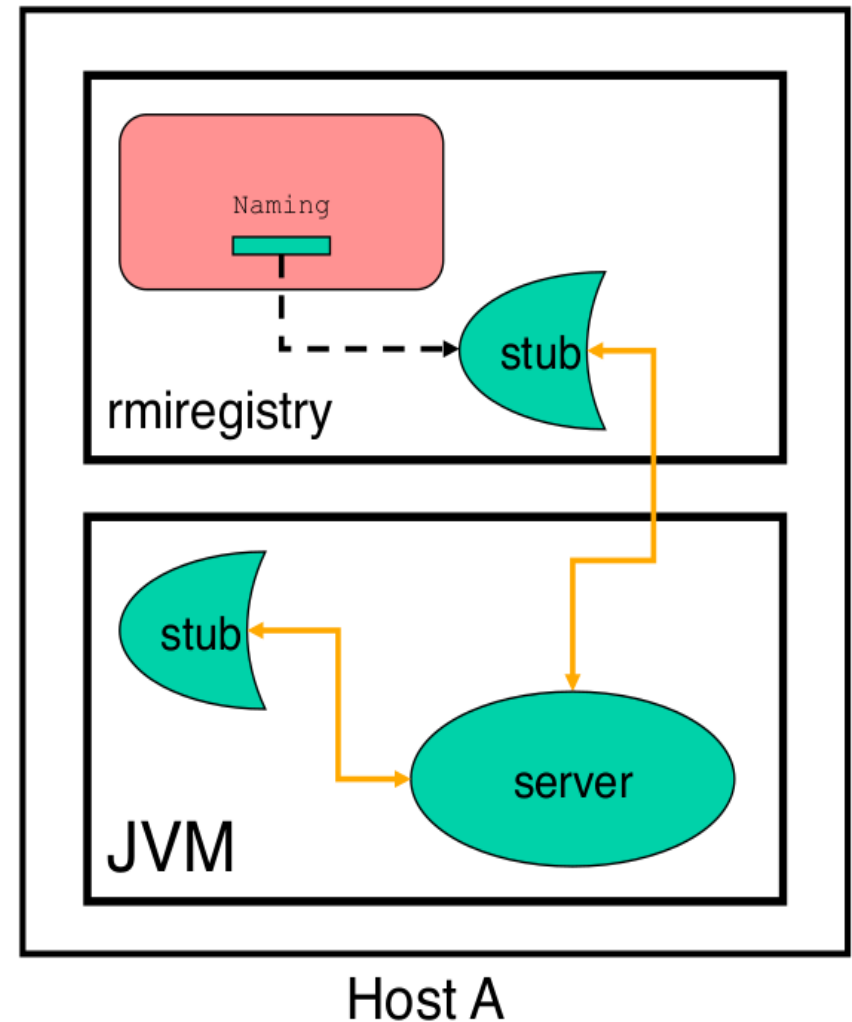
```
Registry registry = LocateRegistry.createRegistry(PORT);
```

Naming = κλάση εναλλακτική
του Registry, σε παλιότερες
υλοποιήσεις



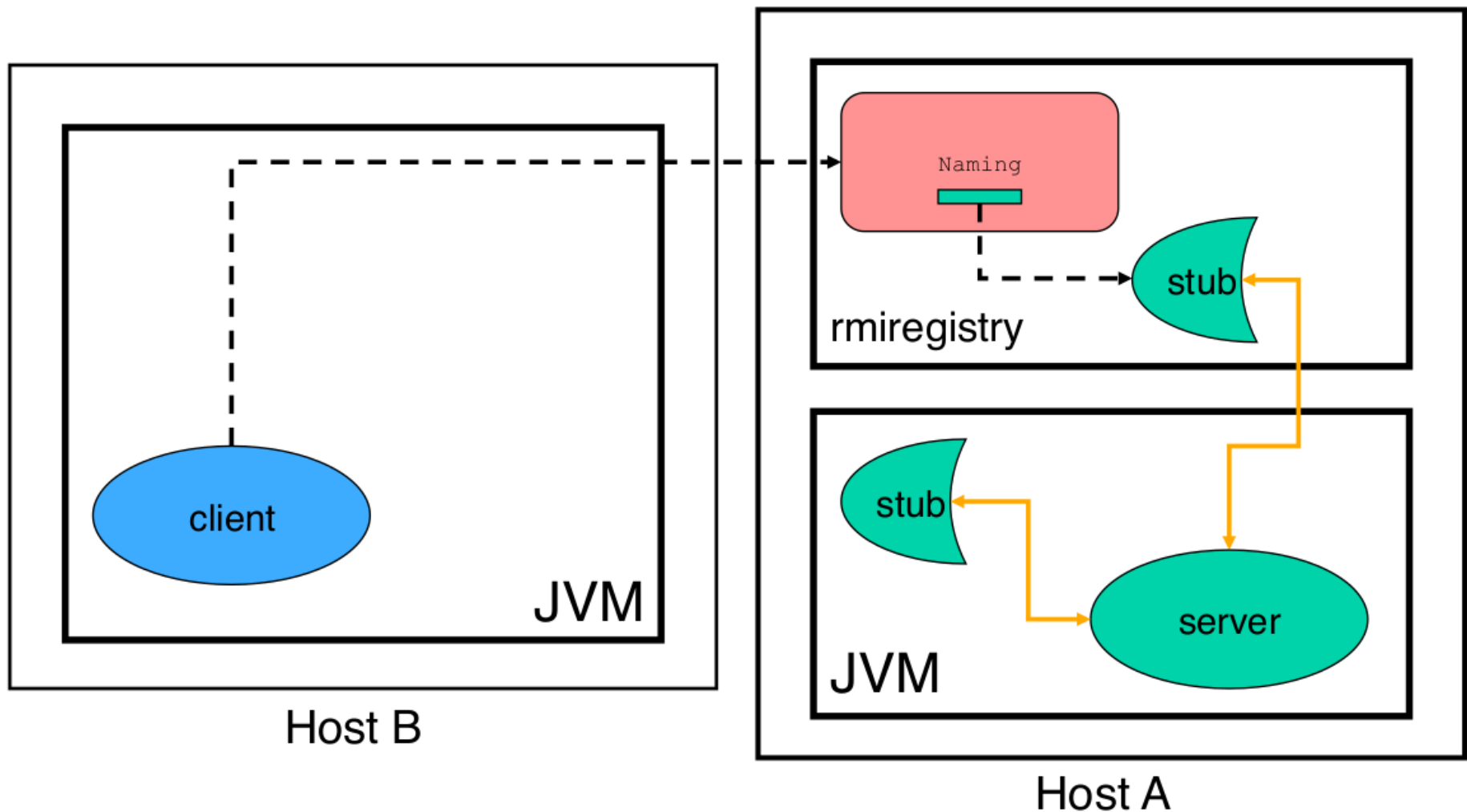
Εκτέλεση παραδείγματος: διακομιστής 3

```
String rmiObjectName = "Addition";  
registry.bind (rmiObjectName,  
robj);
```



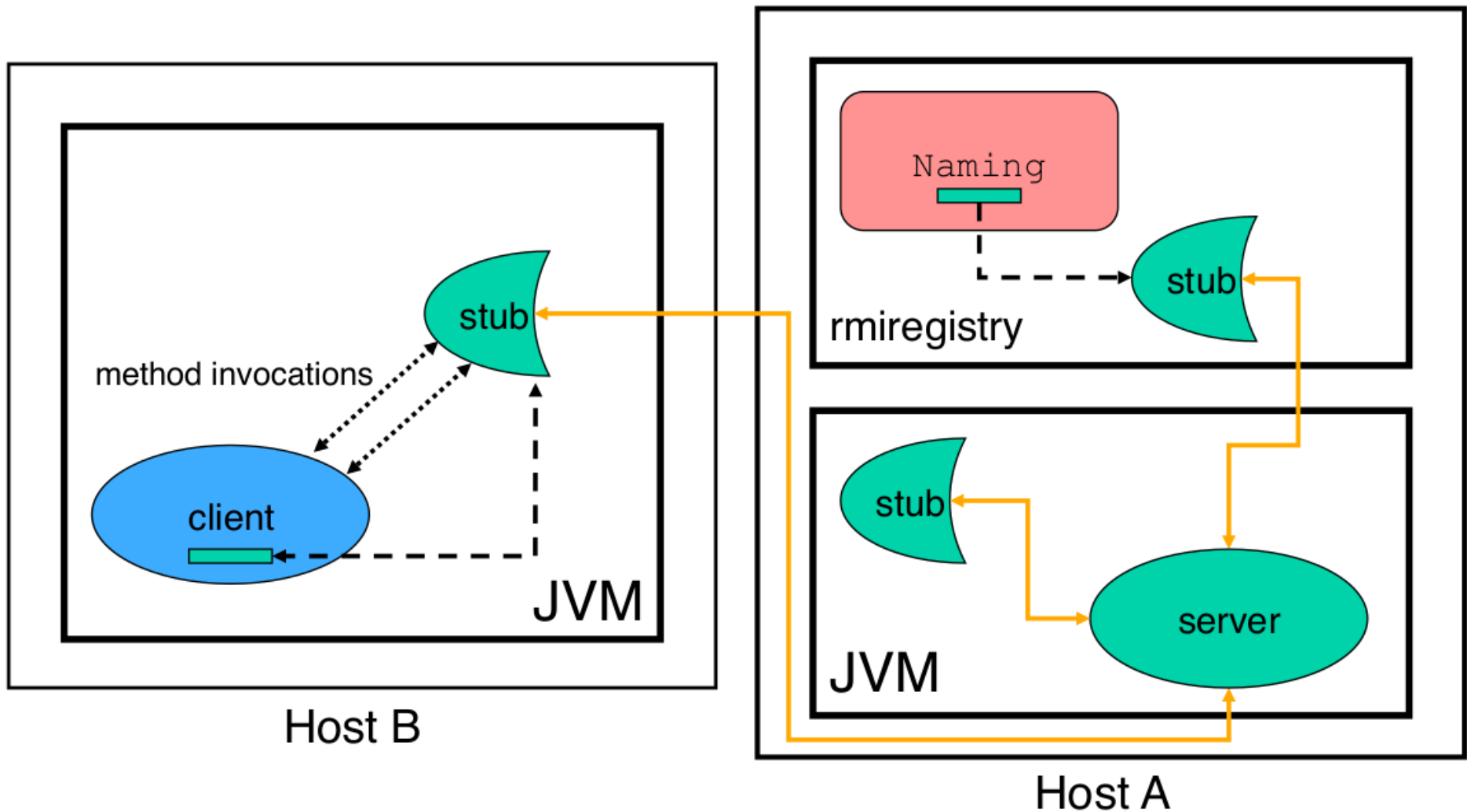
Εκτέλεση παραδείγματος: πελάτης 1

```
Registry registry = LocateRegistry.getRegistry(HOST,  
PORT);
```



Εκτέλεση παραδείγματος: πελάτης 2

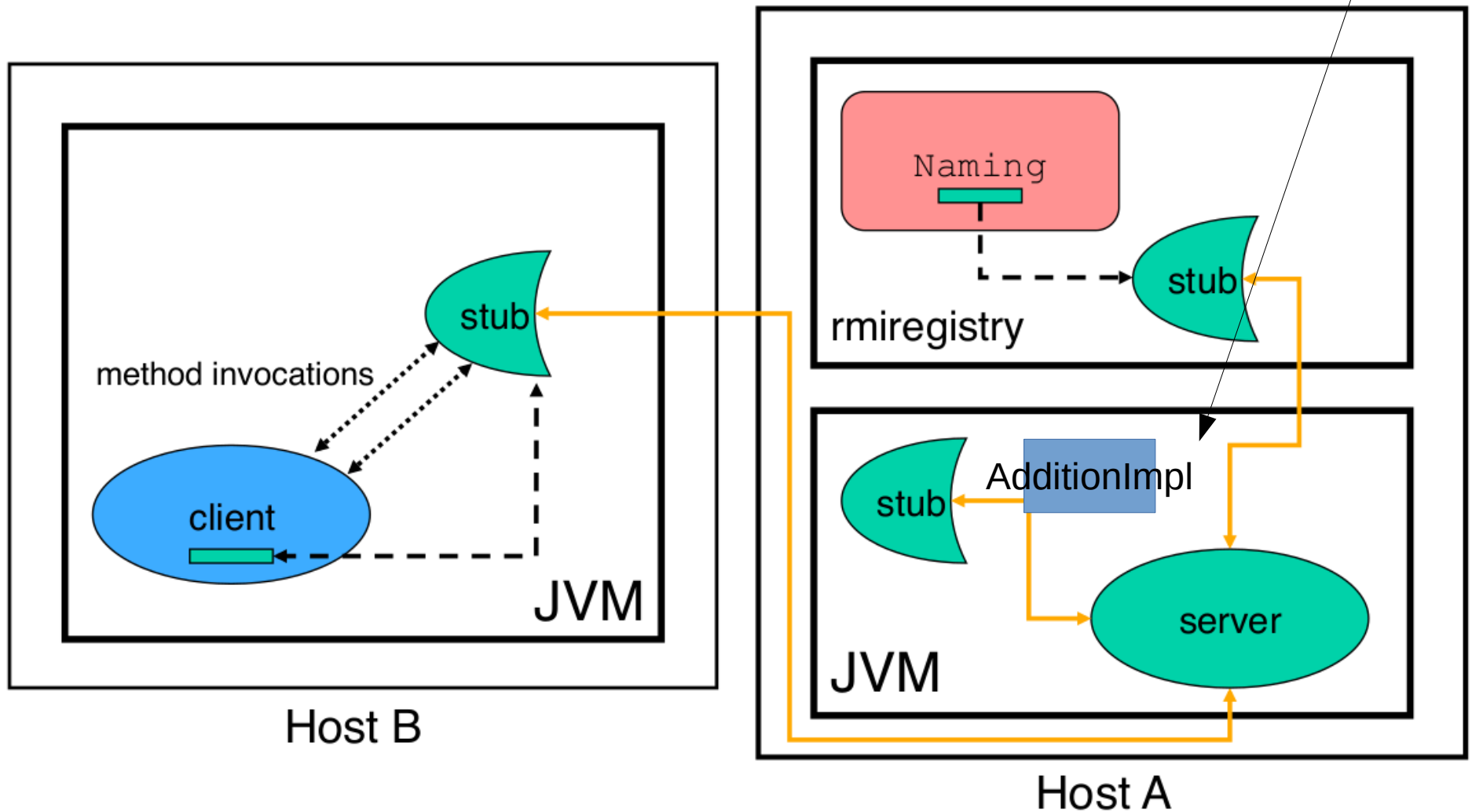
```
String rmiObjectName = "Addition";  
Addition ref = (Addition)registry.lookup(rmiObjectName);
```



Εκτέλεση παραδείγματος: πελάτης 3

```
int result = ref.add (3, 5);  
System.out.println("The result is " +  
result);
```

Δημιουργείται το αντικείμενο
και εκτελείται η μέθοδος



RMI και νήματα

- Ο διακομιστής RMI λειτουργεί ως πολυνηματικός διακομιστής. Κάθε απομακρυσμένη κλήση δημιουργεί ένα νήμα που λειτουργεί σε νέο στιγμιότυπο της υλοποίησης του απομακρυσμένου αντικειμένου.
- Όμως η μπορεί στην υλοποίηση μπορεί να ορίζει μοιραζόμενες μεταβλητές (static).
- Τότε στην υλοποίηση του απομακρυσμένου αντικειμένου απαιτείται χρήση μεθόδων αμοιβαίου συγχρονισμού ή και συγχρονισμού σε συνθήκες.
- Ο διακομιστής RMI ισοδυναμεί με επόπτη του οποίου οι κλήσεις στις δημόσιες διαδικασίες γίνονται απομακρυσμένα.



Γενικευμένη μεταβίβαση μηνυμάτων

Γενικευμένη μεταβίβαση μηνυμάτων

- Το μοντέλο μεταβίβασης μηνυμάτων έχει βρεί εφαρμογή και σε συστήματα μοιραζόμενης μνήμης.
- Οι διεργασίες (ή τα νήματα) εκτελούνται σε συστήματα μοιραζόμενης μνήμης (κοινή φυσική μνήμη) αλλά δεν μοιράζονται μεταβλητές (διακριτοί χώροι διευθύνσεων, απουσία μοιραζόμενων μεταβλητών: αρχιτεκτονική share nothing).
- Το μοντέλο μπορεί σχετικά εύκολα να επεκταθεί σε συστήματα κατανεμημένης μνήμης. Αυτό που αλλάζει είναι η μηχανισμός μεταβίβασης μηνυμάτων. Στην μοιραζόμενη μνήμη έχουμε ουρές, στη κατανεμημένη μνήμη δίκτυο.

Ada rendezvous 1

- Το μοντέλο πελάτη – διακομιστή μεταφέρεται στη μοιραζόμενη μνήμη και ενσωματώνεται στη γλώσσα προγραμματισμού.
- Η γλώσσα διαθέτει task modules (δηλαδή διεργασίες) πελάτη και διακομιστή που επικοινωνούν μεταξύ τους με χρήση ουράς (δεν απαιτείται ρητός ορισμός της ουράς).
- Τα task modules (κυρίως του διακομιστή) μπορεί να διαθέτει μηχανισμό ταυτοχρονισμού, βασισμένο στη πολυπλεξία, όχι σε νήματα.
- Μοιάζει πολύ με RPC αλλά σε σύστημα μοιραζόμενης μνήμης.

task BUFFER is

```
entry STORE(C: in CHARACTER);  
-- Public procedure Put a character into the buffer  
-- Called by producer tasks  
entry RETRIEVE(C: out CHARACTER);  
-- Public procedure Take one out  
-- Called by consumer tasks  
entry PRODUCER_DONE;  
-- Public procedure PRODUCER has shut down  
-- Called by producer tasks
```

end BUFFER;

task body BUFFER **is**

```
-- BUFFER is implemented as a circular queue:  
BUFFER_SIZE: constant INTEGER := 100;  
BUFFER_DATA: array (0..BUFFER_SIZE - 1) of CHARACTER;  
BUFFER_FRONT: INTEGER := 0;  
BUFFER_REAR: INTEGER := 0;  
BUFFER_EMPTY: BOOLEAN := TRUE;  
BUFFER_FULL: BOOLEAN := FALSE;
```



```
-- When the producer tells us it is finished, we must
-- remember this
PRODUCER_IS_DONE: BOOLEAN := FALSE;
```

```
begin
```

```
-- As long as the PRODUCER is still active or we still
-- have data, keep servicing the two other tasks:
```

```
while not (BUFFER_EMPTY and PRODUCER_IS_DONE) loop
```

```
select
```

```
  when not BUFFER_FULL =>
```

```
    accept STORE(C: in CHARACTER) do
```

```
      BUFFER_DATA(BUFFER_REAR) := C;
```

```
      BUFFER_REAR := (BUFFER_REAR + 1) rem BUFFER_SIZE;
```

```
      BUFFER_EMPTY := FALSE;
```

```
      BUFFER_FULL := BUFFER_REAR = BUFFER_FRONT;
```

```
    end STORE;
```

```
or
```

```
...
```

...

```
when not BUFFER_EMPTY =>  
  accept RETRIEVE(C: out CHARACTER) do  
    C := BUFFER_DATA(BUFFER_FRONT);  
    BUFFER_FRONT := (BUFFER_FRONT + 1) rem BUFFER_SIZE;  
    BUFFER_EMPTY := BUFFER_FRONT = BUFFER_REAR;  
    BUFFER_FULL := FALSE;  
  end RETRIEVE;
```

or

```
  accept PRODUCER_DONE do  
    PRODUCER_IS_DONE := TRUE;  
  end PRODUCER_DONE;
```

```
end select;
```

```
end loop;
```

```
-- Tell the CONSUMER that we have no more data to give it  
CONSUMER.NO_MORE_CARDS;
```

```
end BUFFER;
```

Ada rendezvous 2

- Προσέξτε τη δομή

Select, when, accept

που υλοποιεί πολυπλεξία με ένα ταυτόχρονο if / switch. Κάθε φορά μπορεί να εκτελεστεί μόνο ένας κλάδος της δομής select, εάν έχει γίνει η κατάλληλη κλήση από πελάτη (accept) και αν ισχύει η αντίστοιχη συνθήκη when (φρουρός, guard).

- Για πλήρη κώδικα δείτε

https://cs.gordon.edu/courses/cs322/slides/concurrency_ada/producer_consumer.html

Ουρά της Python

- Η Python στα αρθρώματα threading και multiprocessing διαθέτει αντικείμενα ταυτόχρονης Ουράς με βασικές μεθόδους

Get()

Put()

- Οι Ουρές αυτές έχουν λειτουργία ανάλογη με αυτή του Buffer του Παραγωγού Καταναλωτή ή της BlockingQueue της Java.

```
from multiprocessing import *
import time
from random import randint
```

```
def addManyNumbers(numNumbers, q):
    s= 0
    for i in range(int(numNumbers)):
        s= s + randint(1, 100)
    q.put(s)
```

```
def addManyPar():
    totalNumNumbers= 10000000
```

```
q= Queue()
```

```
p1= Process(target=addManyNumbers,
            args=(totalNumNumbers/2, q))
p2= Process(target= addManyNumbers,
            args=(totalNumNumbers/2, q))
```

```
...
```



...

```
startTime= time.time()
```

```
# p1.join()
```

```
# p2.join()
```

```
answerA= q.get()
```

```
answerB= q.get()
```

```
endTime= time.time()
```

```
timeElapsed = endTime-startTime
```

```
print("Time:", timeElapsed)
```

```
print("Sum:", answerA+answerB)
```

```
if __name__ == "__main__":  
    addManyPar()
```

Βιβλιοθήκες Python

- Η Python διαθέτει πολλές χρήσιμες βιβλιοθήκες όπως map, reduce, events, asyncio, futures
- Manager : server για διαμοιρασμό δεδομένων μεταξύ διεργασιών (multiprocessing)
- Επίσης διαθέτει concurrent δομές δεδομένων
- Επιπλέον πολλά εργαλεία για κατανεμημένο υπολογισμό.

Κανάλια CSP 1

Μοντέλο CSP (Hoare, 1985)*:

- Το πρόγραμμα αποτελείται μόνο από ελαφρές διεργασίες (αναλογία με τα σημερινά νήματα).
- Δεν υπάρχουν μοιραζόμενες μεταβλητές (share nothing architecture).
- Κάθε μεταβλητή του προγράμματος ανήκει (είναι ιδιωτική / τοπική) σε κάποια διεργασία.
- Οι μόνες μοιραζόμενες (καθολικές) 'μεταβλητές' είναι τα κανάλια.

*Πρώτη υλοποίηση: γλώσσα Occam και CPU Transputer, 1985, υποστήριξη τοπικής και 'δίκτυακής' επικοινωνίας (της εποχής).

Κανάλια CSP 2

- Κάθε κανάλι ορίζεται στη μεταγλώττιση και συνδέει δύο διεργασίες.
- Τα κανάλια εκτελούν μονόδρομη επικοινωνία: διεργασία αποστολέας και διεργασία παραλήπτης.
- Τα κανάλια μεταβιβάζουν μηνύματα μεταξύ των διεργασιών.
- Τα μηνύματα είναι τιμές μεταβλητών ή εκφράσεων συγκεκριμένου τύπου (πχ `chan int`).
- Τα κανάλια είναι FIFO buffers με χωρητικότητα μιας θέσης. Παραγωγός καταναλωτής με buffer μιας θέσης: αυστηρός συγχρονισμός.

Κανάλια GO 1

- Η γλώσσα Go έχει χρησιμοποιεί το μοντέλο CSP με τροποποιήσεις:
- Κανάλια και διεργασίες μπορεί να δημιουργούνται δυναμικά, όχι μόνο κατά τη μεταγλώττιση.
- Τα κανάλια υποστηρίζουν μόνο τοπική επικοινωνία. Απαιτείται ειδική βιβλιοθήκη για δικτυακή επικοινωνία.
- Τα κανάλια είναι μοιραζόμενα μεταξύ διεργασιών, δηλαδή πολλές διεργασίες μπορούν να γράψουν σε / να διαβάσουν από ένα κανάλι. Στην ουσία τα κανάλια είναι ταυτόχρονες ουρές μηνυμάτων.

Κανάλια GO 2

- Η επικοινωνία μπορεί να είναι σύγχρονη ή ασύγχρονη (unbuffered, buffered FIFO channels). Πολλαπλοί παραγωγοί καταναλωτές με buffer μιας ή πολλών θέσεων: αυστηρός ή χαλαρός συγχρονισμός.
- Ο αυστηρός συγχρονισμός χρησιμοποιείται και ως αναμονή τερματισμού (αντίστοιχο του join).
- Η Go επιτρέπει και μοιραζόμενη μνήμη και προσφέρει και μηχανισμό αμοιβαίου αποκλεισμού (τύπου mutex) αλλά όχι μηχανισμό συγχρονισμού (τύπου wait – notify).


```
import (
    "fmt"
    "time"
)

var done = make(chan bool)           //unbuffered channel, sync
var msgs = make(chan int, 10)        //buffered channel of 10 int

func produce() {
    for i := 0; i < 100; i++ {
        time.Sleep(100 * time.Millisecond)
        msgs <- i                     //send i to msgs- blocks if
                                      //buffer is full
        fmt.Println("Producer: ", i)
    }
    fmt.Println("Before passing true to done")
    done <- true                      //send sync message to done
}
```

Παραγωγός καταναλωτής Go 2

```
func consume() {  
    for {                                     //loops for ever  
        msg := <- msgs                     //receives from channel msgs  
                                           //defines var msg and assigns  
                                           //value from channel  
                                           //blocks on receive  
        fmt.Println("Consumer: ", msg)  
        time.Sleep(100 * time.Millisecond)  
    }  
}  
  
func main() {  
    go produce()                             //fork a producer  
    go consume()                             //fork a consumer  
    <-done                                   //wait for any sync from done  
    fmt.Println("After calling DONE")  
}
```

Παρατηρήσεις 1

- Ο καταναλωτής δεν τερματίζει κανονικά αλλά αναμένουν δεδομένα “για πάντα”. Θα μπορούσε να προβλεφθεί δεδομένο που καθορίζει το τερματισμό.
- Πολλαπλοί παραγωγοί μπορεί να εισάγουν δεδομένα στο ίδιο μοιραζόμενο κανάλι.
- Πολλαπλοί καταναλωτές μπορεί να διαβάζουν δεδομένα από το ίδιο μοιραζόμενο κανάλι μέχρι να εξαντληθούν τα δεδομένα.
- Και στις δύο περιπτώσεις δεν υπάρχει εγγύηση δικαιοσύνης.

Παρατηρήσεις 2

- Το `main` μπορεί να αναμένει την παραλαβή πολλαπλών εγγραφών στο κανάλι `done` ώστε να διασφαλιστεί ο τερματισμός όλων των παραγωγών (ή και καταναλωτών). Υπάρχει ειδική δομή

```
for range ... {  
    <- done  
}
```

που αναμένει αποστολές μηνύματος τερματισμού παραγωγών με οποιαδήποτε σειρά.

Παρατηρήσεις 3

- Υπάρχει η δυνατότητα πολυπλεξίας με υπό συνθήκη παραλαβή αλλά όχι guarded select όπως στην Ada

```
select {  
    case msg1 := <-c1:  
        fmt.Println("received", msg1)  
    case msg2 := <-c2:  
        fmt.Println("received", msg2)  
}
```

- Τα κανάλια μπορούν να περάσουν ως ορίσματα, να δημιουργήσουμε array of channels, ακόμη και channel of channels.

```
...  
const noPhilosophers = 5  
const iterations = 100  
var done = make chan(bool)
```

Γευματίζοντες φιλόσοφοι Go 1

```
func main() {  
  
    //create unbuffered fork channels and put a value in each  
    for i := 0; i < noPhilosophers; i++ {  
        aFork[i] := make(chan int)  
        aFork[i] <- 1  
    }  
  
    //run philosophres in parallel with resource ordering  
    for i := 0; i < noPhilosophers-1; i++ {  
        go philosopher(aFork[i], aFork[i+1], done)  
    }  
    go philosopher(aFork[0], aFork[noPhilosophers], done)  
  
    for i := 0; i < noPhilosophers; i++ {  
        <- done // wait for philosophers to finish  
    } // for range can be used..  
}
```

Γευματίζοντες φιλόσοφοι Go 2

```
func philosopher(oneFork, otherFork chan int,
                 done chan bool) {

    for h := 0; h < iterations; h++ {
        think()
        <- oneFork           // pick up forks: read from channel
        <- otherFork         // if the channel is empty block
        eat()
        otherFork <- 1       // put down forks: write a value
        oneFork <- 1         // so that next has something to read
    }
    done <- true             // send sync
}
```

```
import Data.Char
import Control.Concurrent
import Control.Concurrent.Chan
```

Παραγωγός καταναλωτής Haskell

```
main = do
    c  <- newChan
    cs <- getChanContents c
    forkIO (producer c)
    consumer cs

where
    -- thread one: producer
    producer c = forever $ do
        key <- getChar
        writeChan c key

    -- thread two: consumer
    -- read cs buffer and shift(remove) data from buffer
    -- print data to screen
    consumer = mapM_ print . map shift
        where shift c | isAlpha c = chr (ord c + 1)
                      | otherwise = c
```

-- new channel c to write to
-- read from channel c to cs
-- char producer arg c
-- consumer arg cs

-- read char from keyboard
-- write to channel c

Δράστες (Actors, Hewitt)* 1

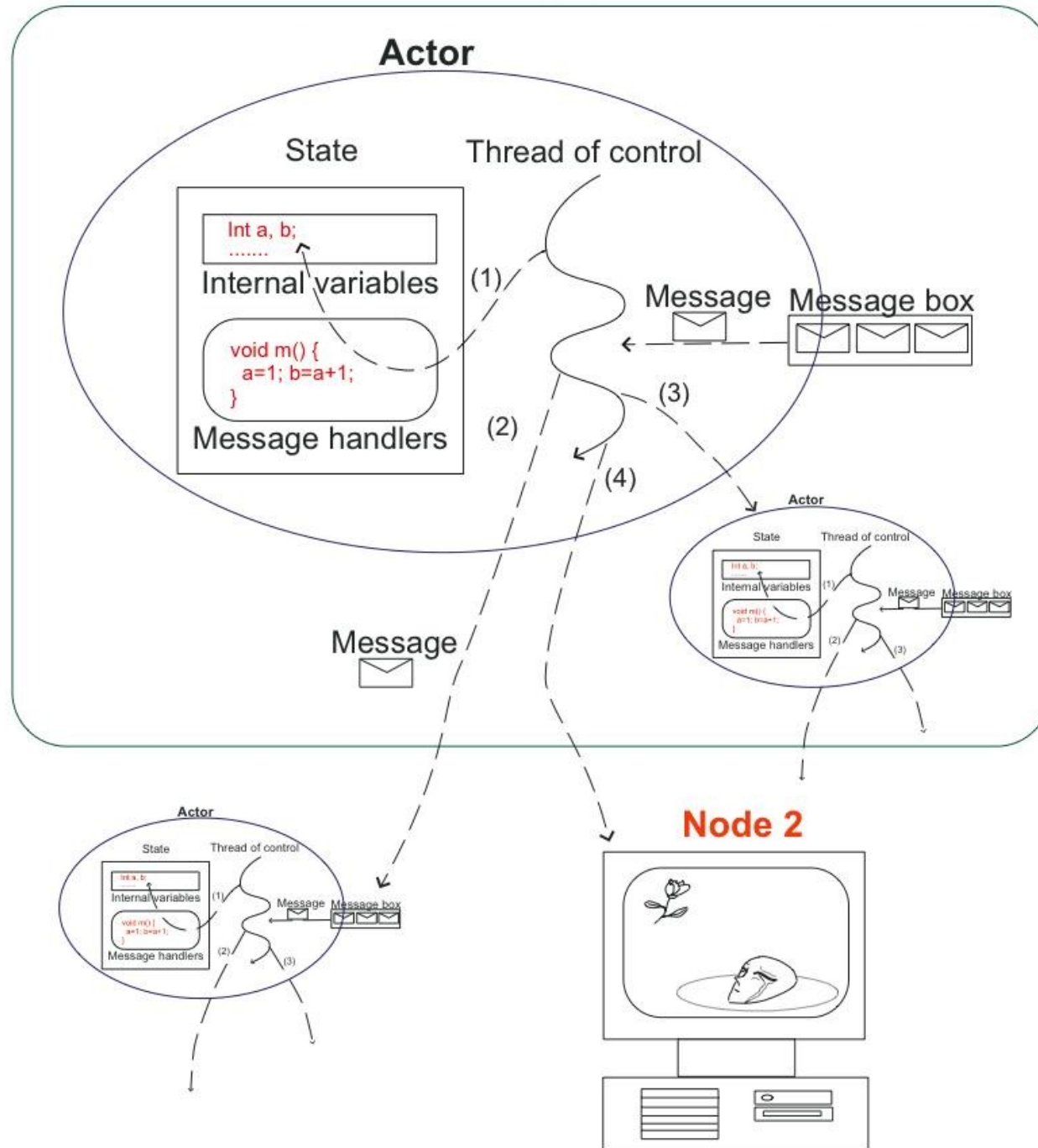
Διακομιστές που η εκτέλεσή τους βασίζεται στην αντίδραση (react) σε εισερχόμενα μηνύματα. Ένας δράστης που δέχεται ένα μήνυμα μπορεί:

- να αλλάξει την κατάστασή του: απλό rendezvous
- να αλλάξει την κατάστασή του και να στείλει μηνύματα σε άλλους δράστες: εκτεταμένο rendezvous ή αίτημα – απάντηση
- να δημιουργήσει νέους δράστες (αντίγραφα του συνήθως)
- να μεταναστεύει σε άλλο κόμβο (αν βρίσκεται σε κατανεμημένο υπολογιστικό σύστημα).

*https://en.wikipedia.org/wiki/Actor_model

Node 1

Δράστες 2



Erlang απλοποιημένη

```
run() ->
    S = spawn(counter, [0]),
    spawn(send_msgs(S, 100000)),
    S.
```

```
counter(Sum) ->
    Receive
        {inc, Amount} -> counter(Sum+Amount),
        value -> io:fwrite("Value is ~w~n", [Sum]);
    end.
```

Δράστης

```
send_msgs(S, 0) -> true;
send_msgs(S, Count) ->
    S ! {inc, 1},
    send_msgs(S, Count-1).
```

Αναδρομή

```
...
class Counter extends Actor {
  var counter: Int = 0;
  def act() = {
    while (true) {
      receive {
        case Inc(amount) =>
          counter += amount
        case Value =>
          println("Value is "+counter)
          exit()
      }
    }
  }
}

object ActorTest extends Application {
  val counter = new Counter
  counter.start()
  for (i <- 0 until 1000000) {
    counter ! Inc(1)
  }
  counter ! Value
  // Output: Value is 1000000
}
```

Scala
απλοποιημένη

Scala παραγωγός καταναλωτής 1

```
object ProducerConsumer
{
    case class Message(msg : String)

    def main(args : Array[String]) : Unit =
    {

        // Spawn Producer
        spawn
        {
            val importantInfo : Array[String] = Array(
                // strings to send...
            );

            importantInfo.foreach ((msg) => consumer ! msg)
            consumer ! "DONE"
        }
    }
}
```

Scala παραγωγός καταναλωτής 2

...

```
// Spawn consumer  
spawn  
{
```

```
  val consumer =  
    actor  
    {
```

```
      var done = false  
      while (!done)  
      {
```

```
        receive  
        {
```

```
          case msg =>
```

```
            System.out.println("Received -> " + msg)  
            done = (msg == "DONE")
```

```
        }
```

```
      }
```

```
    }
```

```
  }
```

```
}
```

```
}
```

Θα μπορούσε να γραφεί και
producer ? msg

